Data Analytics Project: Kc_house_data

Team Members:

1. Harsha Vardhan Gone
2. Namratha Reddy Annem
3. Neeli Ajay Kumar
4. Pradeep Kumar
5. Lavanya Koppula

Dataset Used: Ks_house_data**.**

This document outlines a structured, end-to-end data analysis project based on ks house data. The project is divi **Data Analytics Project: Ks_house_trends.**

This document outlines a structured, end-to-end data analysis project based on Ks_house_data.

Dataset: Ks_house_data**.** (Assumed to be in a CSV file)

The **primary libraries/modules** required for this project are:

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (np.diff).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Stacked Bar Charts).

🎯 Project Coverage

Category	Skills Covered
Data Preprocessing	Data Loading, Missing Value Handling.
Pandas Operations	Filtering, Selection, Aggregation, Feature Engineering
NumPy Calculations	Yearly growth calculation (np.diff)
Visualization	Line Graph, Bar Chart, Pie Chart, Stacked Bar Chart

Scenario 1: Data Loading & Basic Cleaning

Description:

In this scenario, we work with a house sales dataset. First, we load the data using Pandas. Then, we look at the first few rows to understand the data. Then, we look at the first few rows to understand the data. We check if there are any missing values in important columns. After that, we fill missing values using mean or mode. Finally, we make sure all columns have the correct numeric format.

Task	Description
1.	Load the dataset using Pandas.
2.	Display the first 5 rows and column names to understand structure.
3.	Check for missing values (bedrooms, bathrooms, sqft_living, price).
4.	Fill missing values: (bedrooms → use mode, bathrooms → use mean, sqft_living → use mean, price → use mean)
5	Convert these columns to numeric if required: (bedrooms, bathrooms, sqft_living, price)

Output

The first part of the output shows the first five rows of your dataset. This is simply a preview to help you understand what the data looks like, including values such as house prices, dates, locations, and sizes. dataset contains a total of 21 columns (features), but only the first 5 rows are being displayed for quick inspection. These include important features like price, number of bedrooms and bathrooms, square footage, location details, and construction or renovation years. The final section with values equal to 0 represents the count of missing values in selected columns such as bedrooms, bathrooms, sqft_living, and price. Since all the values are 0, it means there are no missing (null) entries in these columns.

```
First 5 rows:
           id             date     price  ...     long  sqft_living15  sqft_lot15
0  7129300520  20141013T000000  221900.0  ... -122.257           1340        5650
1  6414100192  20141209T000000  538000.0  ... -122.319           1690        7639
2  5631500400  20150225T000000  180000.0  ... -122.233           2720        8062
3  2487200875  20141209T000000  604000.0  ... -122.393           1360        5000
4  1954400510  20150218T000000  510000.0  ... -122.045           1800        7503

[5 rows x 21 columns]
==========================================================

Column Names:
Index(['id', 'date', 'price', 'bedrooms', 'bathrooms', 'sqft_living',
       'sqft_lot', 'floors', 'waterfront', 'view', 'condition', 'grade',
       'sqft_above', 'sqft_basement', 'yr_built', 'yr_renovated', 'zipcode',
       'lat', 'long', 'sqft_living15', 'sqft_lot15'],
      dtype='str')
==========================================================
bedrooms       0
bathrooms      0
sqft_living    0
price          0
dtype: int64
==========================================================
==========================================================
```

Conclusion:

In this scenario, we cleaned the dataset step by step. We handled missing values properly using mean and mode. We also checked and fixed data types. Now the dataset is clean and ready for analysis. This helps to get correct results in future steps.

Scenario 2: Line Graph + Save

Description:

This scenario mainly focuses on analyse and visualizing house price data using a small subset of the dataset. The primary goal is to select the relevant fields, specifically the house ID and price, and limit the analysis for first 10 records. The price values are converted into a NumPy array and plotting easier. A line graph created with index (0–9) on the X-axis and price on the Y-axis to observe how prices vary across these entries.

Tasks	Description
1	Select **id** and **price** from the dataset.
2	Take the first 10 rows only.
3	Convert **price** into a NumPy array.
4	Plot a line graph: ○ X-axis → index (0–9) ○ Y-axis → Price
5	Add title and labels: ○ Title ○ X-label ○ Y-label
6	Save the graph: plt.savefig ("house_prices_line.png")

Output:

Here, I am displaying the selected columns id and price from the dataset which will be focusing on the first 10 records. This output shows a sample of house prices associated with their respective IDs. The purpose is to observe how the price values vary across these entries, which will be further used for visualization in the form of a line graph.

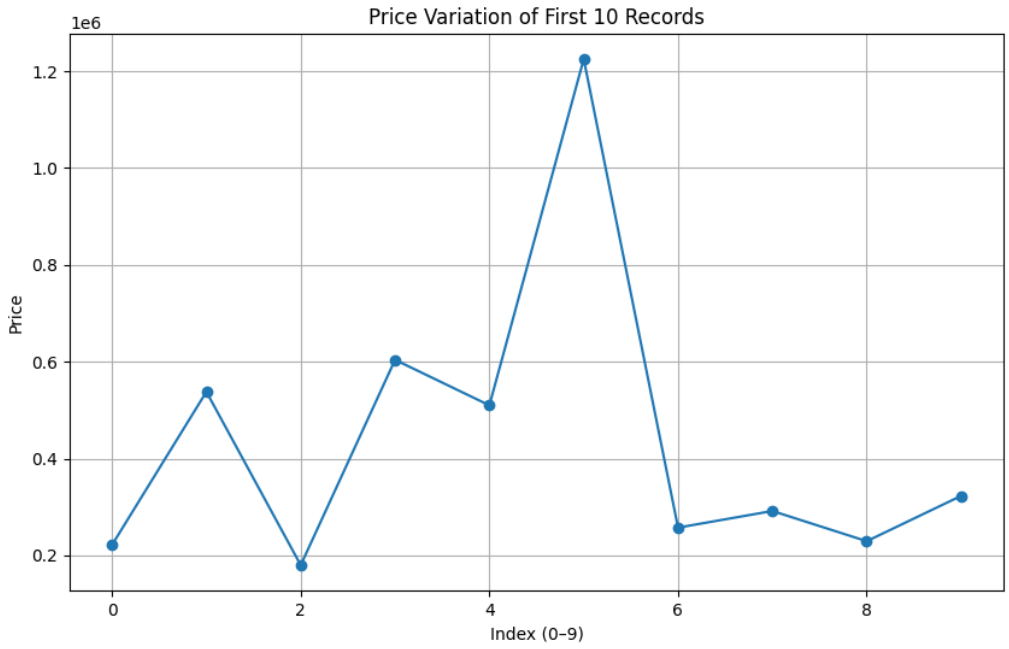
```
(S2) Selecting required columns
                id     price
0       7129300520  221900.0
1       6414100192  538000.0
2       5631500400  180000.0
3       2487200875  604000.0
4       1954400510  510000.0
...            ...       ...
21608    263000018  360000.0
21609   6600060120  400000.0
21610   1523300141  402101.0
21611    291310100  400000.0
21612   1523300157  325000.0

[21613 rows x 2 columns]
============================================================

(S2) Data (first 10 rows)
           id      price
0  7129300520   221900.0
1  6414100192   538000.0
2  5631500400   180000.0
3  2487200875   604000.0
4  1954400510   510000.0
5  7237550310  1225000.0
6  1321400060   257500.0
7  2008000270   291850.0
8  2414600126   229500.0
9  3793500160   323000.0
```

Graph:

The line graph is plotted with index on the X-axis and price on the Y-axis. It shows the prices for first 10 records, where each point represents the price of a house and the lines connecting them illustrate how the prices increase or decrease across these entries. This helps in easily identifying variations and trends in house prices within the selected sample.



Conclusion:

The graph clearly shows that house prices may vary from one record to another record. Some houses are priced much higher, while the others are lower. This indicates that prices are not consistent across the dataset. This visualization makes it easy to compare the prices and quickly understand the differences among the first 10 records.

Scenario 3: Filtering + Bar Chart + Save

Description:

We first select only houses that cost more than 10,00,000 to focus on expensive properties. Then we count how many houses fall into each bedroom category. From these, we pick the most common bedroom types. The data is converted into arrays and shown using a simple bar chart, where bedrooms are on the X-axis and counts on the Y-axis.

Tasks	Description
1.	Filter houses where: ○ price > 1000000
2.	Count number of houses per: ○ bedrooms
3.	Select top bedroom categories.
4.	Convert results to NumPy arrays
5.	Plot a bar chart: ○ X-axis → Bedrooms ○ Y-axis → Count
6.	Rotate labels if needed.
7.	Save graph: plt.savefig("expensive_houses_bar.png")

Graph:

- The bar chart shows the number of expensive houses (price > 1,000,000) across different bedroom categories.
- It highlights that 4-bedroom houses are the most common among high-priced properties, followed by 3 and 5 bedrooms.

Conclusion:

The chart helps quickly understand which bedroom sizes are most common among expensive houses. This makes it easier to spot trends, like whether bigger homes dominate high prices. It also simplifies decision-making for analysis or real estate insights. Overall, visualizing filtered data makes patterns much clearer.

Scenario 4: Pie Chart (Bedroom Distribution) + Save

Description:

The pie chart shows the distribution of houses based on the number of bedrooms. It is created using the top 5 most common bedroom categories in the dataset. Each part of the pie represents a bedroom category, and the size of each part shows how many houses belong to that category. The percentage labels help us clearly understand the share of each category in the total dataset.

Tasks	Description
1.	Counting numbers of houses by: **bedrooms**
2.	Selecting Top 5 bedroom categories
3.	Preparing **Labels** and **Values**
4.	Plotting **Pie Chart**
5.	Adding percentage labels
6.	Saving graph as plt.savefig("bedroom_distribution.png")

Output Screen:

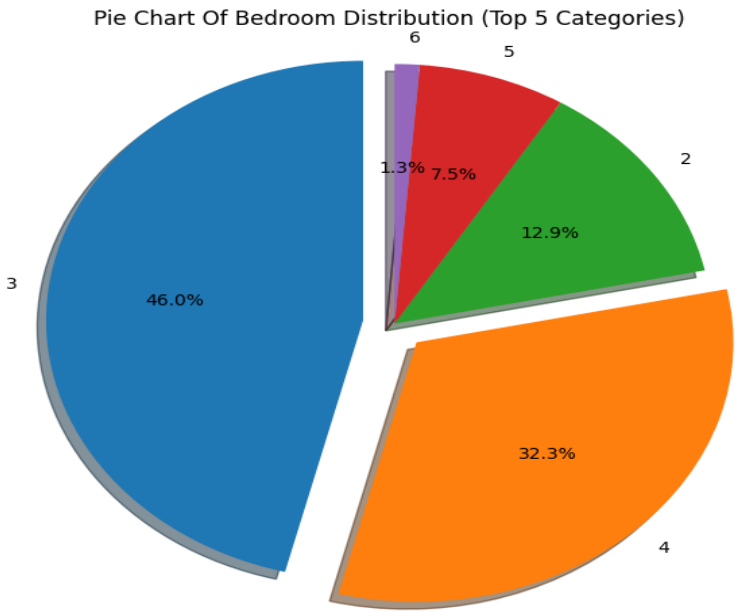
- Here, I'm counting the number of houses and selecting top 5 from the bedrooms.

```
(S4)counting number of houses bedrooms bedrooms
3     9824
4     6882
2     2760
5     1601
6      272
1      199
7       38
0       13
8       13
9        6
10       3
11       1
33       1
Name: count, dtype: int64
============================================================

(S4)Selecting Top 5 bedroom categories bedrooms
3    9824
4    6882
2    2760
5    1601
6     272
Name: count, dtype: int64
```

Graph:

Pie Chart: The pie chart shows how houses are distributed based on the number of bedrooms. It helps us easily see which bedroom types are more common in the dataset.



Conclusion:

From the chart, we can understand that some bedroom categories are more common than others. Houses with a moderate number of bedrooms appear most frequently compared to very small or very large houses. This shows that most people prefer medium-sized houses. Overall, the data is not evenly spread, and a few categories dominate the distribution

Scenario 5: Advanced Analysis + Multiple Graphs

Description:

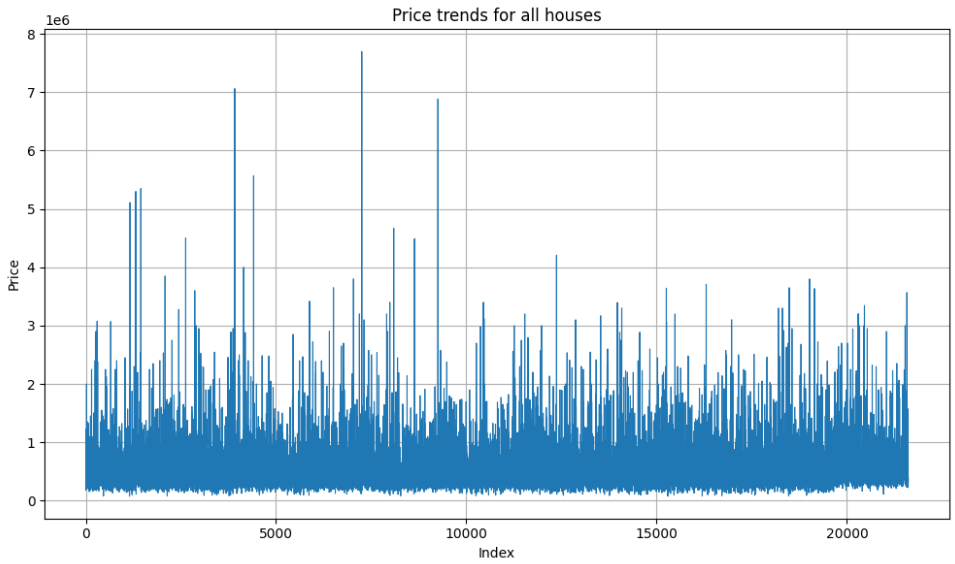
This scenario focuses on advanced analysis of the house sales data by combining feature creation, NumPy calculations, and multiple visualizations. A new column called Price Category is created to group houses into Luxury, Mid Range, and Affordable. NumPy is used to calculate price differences between consecutive houses to understand price variation. Three different graphs including a line graph, stacked bar chart, and histogram are used to visually represent the data. Finally, meaningful insights are drawn from the analysis to understand price patterns and distribution across the dataset.

Task	Description
1.	Create a new column named “Price category” Where, price >= 1000000 → "Luxury" 500000 – 999999 → "Mid Range" < 500000 → "Affordable"
2.	Convert price column to a Numpy array. Calculate Price differences using np.diff()
3.	Plot: a)Line graph: Plot price trend for all houses b)Stacked bar chart: To show count of Price Category per Bedrooms. c)Histogram: Plot distribution of price
4.	Save all graphs using plt.savefig() plt.savefig("price_trend.png") plt.savefig("price_category_stacked.png”) plt.savefig(“price_histogram.png")
5.	Insights: 1. Bedroom category that has the most expensive houses. 2. most common price category. 3. Distribution pattern of house prices? ○ Right-skewed? ○ Normal? ○ Concentrated in a lower price range?

Graph:

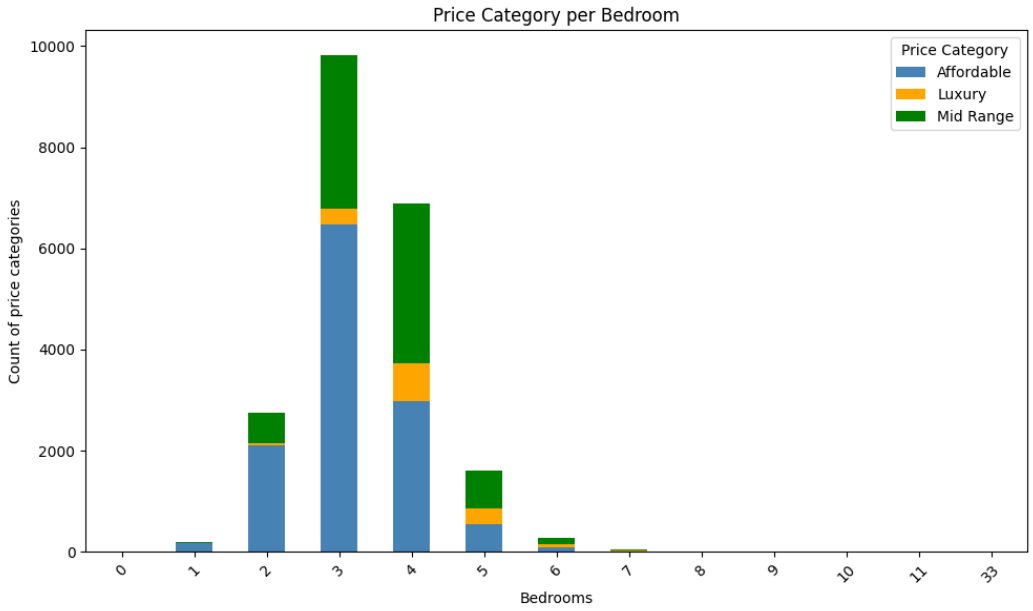
Line Graph:

A line graph is plotted using the house records to visualize how prices vary. x-axis represents the house index and the y-axis shows the corresponding price value.



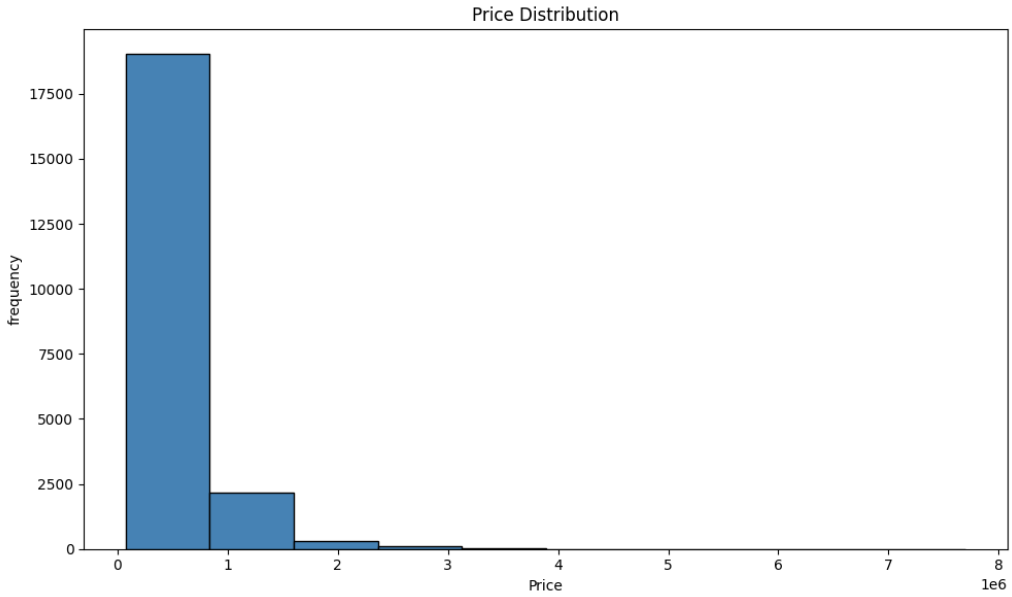
Stacked bar graph:

A stacked bar chart is plotted to show how many houses of each Price Category exist for every bedroom count. This makes it easy to visualize the mix of Affordable, Mid Range, and Luxury houses across different bedroom groups.



Histogram:

A histogram is plotted to visualize how house prices are spread across the dataset. Each bar represents a price range, and the height of the bar shows how many houses fall within that range.



Conclusion:

This scenario helps us understand the house sales data in a much better way by combining feature creation, NumPy, and visualizations. Adding the Price Category column made it easy to group houses into Affordable, Mid Range, and Luxury. The np.diff() function showed how much prices vary between consecutive records.

Each graph gave a different view of the data. The line graph showed prices are irregular, the stacked bar chart confirmed Affordable houses dominate every bedroom count, and the histogram proved that most houses are priced low with very few expensive ones on the right tail.

Final Conclusion:

This project took us through the complete process of analysing a real-world house sales dataset from start to finish. We started with the basics like loading the data and fixing missing values, then slowly moved into filtering, grouping, and finally creating our own features and graphs. By the end, we had a clear picture of how house prices are distributed, which bedroom categories are most common, and how price categories like Affordable, Mid Range, and Luxury are spread across the dataset.

What you have learned

Through this project, we learned how to load and clean a dataset using pandas, handle missing values using mean and mode, and convert columns to the right data types. We used Numpy to convert columns into arrays and calculate differences between values using np.diff().

We also learned how to create different types of graphs using matplotlib, line graphs to show trends, bar charts to compare categories, pie charts to show distribution, stacked bar charts to compare multiple groups at once, and histograms to understand data distribution.

Most importantly, we learned how to read graphs and identify real insights from them like identifying skewness, spotting which category dominates, and understanding what the data is actually about.